

Industrial IoT Sensor Cleaning & Equipment Classifier Pipeline

A Resilient End-to-End ML Pipeline for Cleaning Industrial Sensor Telemetry and Predicting Pump Equipment Failure

AI / ML Project Case Study

Submitted by: Kantapalli Sobhana Aishwarya

Roll Number: A23126510033

Department: Computer Science and Engineering

College: Anil Neerukonda Institute of Technology and Sciences

Date: 17 August 2026

Table of Contents

Table of Contents.....	1
Executive Summary	4
1. Introduction.....	5
1.1 Background and Domain Context.....	5
1.2 Existing Challenges in Pump Failure Detection	5
1.3 Motivation and Role of AI/ML.....	5
2. Problem Statement	7
3. Objectives	8
4. Proposed Solution	9
4.1 Input	9
4.2 Processing	9
4.3 Machine Learning Component.....	9
4.4 Output	9
4.5 User Interaction / Application Layer	9
5. System Architecture.....	10
5.1 Data Layer.....	10
5.2 Preprocessing Layer.....	10
5.3 Modelling Layer.....	10
5.4 Serving / Prediction Layer	10
5.5 Presentation Layer	10
5.6 Deployment Layer	10
6. Dataset Description.....	11
6.1 Categorical Feature Detail	12
7. Data Preprocessing.....	13
7.1 Missing-Value Handling.....	13
7.2 Duplicate Removal.....	13
7.3 Feature Scaling.....	13
7.4 Categorical Encoding.....	13
7.5 Outlier Handling, Class Balancing, and Feature Selection	14
8. Exploratory Data Analysis	15
8.1 Descriptive Statistics for Representative Sensors	16
9. Feature Engineering and Feature Extraction.....	18
9.1 Time-Feature Engineering	18

9.2 Target Derivation	18
9.3 Feature Selection and Encoding.....	18
10. Machine Learning Methodology	19
10.1 Logistic Regression.....	19
10.2 Random Forest	19
10.3 Gradient Boosting	20
10.4 Candidate Model Configuration Summary	21
11. Model Training	22
11.1 Train / Test Split	22
11.2 Validation Strategy	22
11.3 Hyperparameter Tuning	22
11.4 Class Balancing and Feature Scaling	22
11.5 Training Procedure and Artefacts	22
12. Model Evaluation Methodology	24
12.1 Point Metrics	24
12.2 Confusion Matrix	24
12.3 Classification Report.....	24
12.4 ROC Curve.....	24
13. Results.....	25
13.1 Cross-Validation Comparison (Training Split).....	25
13.2 Held-Out Test Evaluation (Selected Model: Logistic Regression).....	26
13.3 Confusion Matrix	26
13.4 ROC Curve.....	27
13.5 Classification Report.....	28
13.6 Feature Importance (Selected Model: Logistic Regression Coefficients)	28
14. Model Comparison and Selection	30
15. System Implementation	31
15.1 Technology Stack.....	31
16. Application Workflow	32
16.1 Dashboard Pages.....	32
16.2 Resilience to Malformed Input	33
17. Key Findings.....	34
18. Challenges Encountered.....	35
18.1 Severe Class Imbalance	35
18.2 Missing and Corrupted Sensor Data	35
18.3 Prediction-Time Input Robustness.....	35

18.4 No Access to the Original Kaggle Dataset.....	35
19. Limitations	36
20. Future Enhancements.....	37
21. Conclusion	38
22. References.....	39

Executive Summary

Unplanned failure of industrial water pumps disrupts production, increases maintenance cost, and creates safety risk, yet plant operators typically discover a failing pump only after it has already entered an abnormal (“BROKEN” or “RECOVERING”) state. This project implements an end-to-end machine learning pipeline that predicts equipment failure from multi-sensor industrial telemetry, with a specific engineering focus on tolerating the missing and corrupted sensor readings that are routine in real IoT deployments rather than crashing on them.

The system is built around a schema modelled on the public Kaggle “Pump Sensor Data” dataset (timestamp, 30–52 sensor_XX numeric readings, a machine_status column). Because the real Kaggle CSV was not included in the provided project materials, the model reported here was trained and evaluated on the project's own schema-compatible synthetic telemetry generator (data/raw/pump_sensor_data.csv, 20,000 rows), which simulates a hidden equipment-risk process, five “critical” drifting sensors, and missing-completely-at-random sensor dropout. The binary target equipment_failure is derived from machine_status (NORMAL = 0; RECOVERING and BROKEN = 1).

A single scikit-learn ColumnTransformer (median imputation and standard scaling for 33 numeric features; most-frequent imputation and one-hot encoding for 2 categorical features) feeds three candidate classifiers - Logistic Regression, Random Forest, and Gradient Boosting - compared with stratified 5-fold cross-validation across accuracy, precision, recall, F1-score, and ROC-AUC. Logistic Regression was automatically selected as the best model by mean cross-validated F1-score (0.3798) and, on a held-out 4,000-row test split it had never seen, achieved 91.95% accuracy, 83.20% recall and 93.95% ROC-AUC on the minority failure class, at the cost of a comparatively low 25.68% precision. The trained pipeline, its comparison and evaluation reports, and a resilient prediction module are served through a 7-page Streamlit dashboard supporting single-record and batch CSV failure prediction.

The project demonstrates a technically sound, leakage-free ML workflow and an unusually strong focus on production-style input resilience, evidenced by dedicated bug-fix commits that hardened the prediction module against partial sensor corruption and infinite values. It should be read as a well-engineered prototype and demonstrator rather than a validated production system: it has not been evaluated on real plant telemetry, uses fixed (non-tuned) hyperparameters, and - as is common for recall-oriented failure detectors - currently trades a high false-positive rate for high recall.

1. Introduction

1.1 Background and Domain Context

Project brief: “Factory sensors frequently disconnect or send corrupted data, causing live ML systems to crash. You will create an end-to-end data processing pipeline that automatically fills in missing sensor readings, scales live telemetry, and predicts equipment failures reliably without runtime errors.”

This is the original problem brief this project was built to satisfy, and it frames the two design priorities that recur throughout this case study: (i) an automated pipeline that cleans - imputes and scales - messy live sensor telemetry, and (ii) an equipment-failure classifier that keeps serving predictions reliably instead of crashing when it meets disconnected or corrupted sensors.

Industrial pumps are core rotating equipment in water treatment, oil and gas, chemical processing, and manufacturing plants. Modern installations increasingly instrument these pumps with Industrial Internet of Things (IIoT) sensors that continuously report readings such as vibration, pressure, flow, and temperature. This telemetry, together with an operational status label, forms the basis for predictive-maintenance systems that aim to flag deteriorating equipment before it fails outright, as opposed to traditional reactive maintenance (fix it after it breaks) or fixed-interval preventive maintenance (service it on a calendar, whether it needs it or not).

This project targets the specific schema of the widely used Kaggle “Pump Sensor Data” telemetry dataset: a timestamp, dozens of sensor_XX numeric channels, and a machine_status column with three states - NORMAL, RECOVERING, and BROKEN. The RECOVERING state represents the pump's return to normal operation after a failure event, and is treated as part of the abnormal condition for this project's binary classification framing (see Section 6).

1.2 Existing Challenges in Pump Failure Detection

Two challenges recur in real deployments of sensor-based failure detection and motivate specific design choices in this project:

- Class imbalance - failures are, by definition, rare events. In this project's dataset only 3.12% of records correspond to an abnormal (RECOVERING or BROKEN) state, so a naive classifier that always predicts “Normal” would already be 96.9% accurate while catching zero real failures.
- Unreliable sensor data - IoT sensors disconnect, drop readings, or send corrupted/overflow values. A model or serving pipeline that assumes clean input will crash or silently mis-predict the moment a real deployment encounters a missing or malformed reading.

Both challenges are addressed directly in the implementation: class imbalance through `class_weight="balanced"` on two of the three candidate models and through stratified sampling everywhere the data is split; unreliable sensor data through a fitted scikit-learn imputation pipeline and a prediction layer that explicitly coerces malformed input to NaN instead of raising an exception (Sections 7 and 18).

1.3 Motivation and Role of AI/ML

A rule-based threshold system (e.g., “alert if sensor_04 exceeds X”) does not generalise well when failure signatures are distributed across many correlated sensors and depend on operating context (operation mode, pump station, time of day). Supervised machine learning offers a way to learn this

multivariate failure signature directly from historical labelled telemetry. This project's motivation is therefore twofold: first, to build and fairly compare several standard classifiers for this task under a leakage-free evaluation protocol; and second, to demonstrate that the resulting model can be served through an interactive dashboard in a way that degrades gracefully rather than failing outright when it meets the imperfect data that is normal in industrial settings.

2. Problem Statement

Given a stream of multivariate sensor telemetry from an industrial water pump - potentially containing missing or corrupted readings - the problem addressed by this project is to classify, at each recorded time step, whether the pump is operating normally or is in an equipment-failure-related state (actively broken, or recovering from a recent failure).

The specific difficulty is that the two classes are highly imbalanced (failure-related records make up only 3.12% of the dataset used in this project), that sensor readings are frequently missing (3.98% of sensor cells in the working dataset), and that any model trained under laboratory conditions must still be able to score a single incomplete reading submitted through a live dashboard without crashing. Plant operators and maintenance engineers - the affected stakeholders - need an early, reasonably reliable signal of failure risk so that maintenance can be scheduled proactively; a manual, sensor-by-sensor review of dozens of channels per pump does not scale to that need, which is why an automated, data-driven classifier is a useful complement to manual inspection.

3. Objectives

1. To build a reusable data-loading and target-derivation pipeline that reads raw pump telemetry, engineers time-based features from the timestamp, and derives a binary equipment_failure label from the raw machine_status column.
2. To design a preprocessing pipeline (scikit-learn ColumnTransformer) that imputes missing numeric and categorical values and scales/encodes features, fitted only on training data to prevent data leakage.
3. To train and fairly compare multiple classification algorithms - Logistic Regression, Random Forest, and Gradient Boosting - using stratified k-fold cross-validation across several metrics.
4. To automatically select the best-performing candidate model by a defined selection metric (F1-score) and evaluate it once on a held-out test split it never saw during training or model selection.
5. To persist the trained pipeline and its evaluation artefacts (comparison results, evaluation report, feature metadata) so they can be reused without retraining.
6. To implement a prediction module that is resilient to missing fields, unparseable values, and infinite values in both single-record and batch-CSV inference, rather than raising unhandled exceptions.
7. To expose the full pipeline - dataset statistics, preprocessing summary, model comparison, evaluation results, feature importance, and live prediction - through an interactive Streamlit dashboard.

4. Proposed Solution

The proposed solution is an end-to-end supervised classification pipeline, implemented in Python with scikit-learn, that turns raw sensor telemetry into a binary failure prediction with an associated probability, and exposes that pipeline through a web dashboard.

4.1 Input

A CSV of pump telemetry: one timestamp column, up to 30 numeric sensor_XX columns, two categorical operational columns (operation_mode, pump_station), and either a raw machine_status column or an already-binary equipment_failure column.

4.2 Processing

Raw timestamps are converted into hour_of_day, day_of_week, and is_weekend features; the target label is derived if not already binary; a fitted ColumnTransformer imputes missing values and scales/encodes the remaining 35 features (Section 7).

4.3 Machine Learning Component

Three candidate classifiers are trained and compared under identical stratified cross-validation; the candidate with the best mean F1-score is refit on the full training split and persisted with Joblib (Sections 10–11).

4.4 Output

For a single reading or an uploaded batch, the persisted pipeline returns a binary prediction (Normal / Failure) and a failure probability in $[0, 1]$, obtained via `predict_proba()` with a 0.5 decision threshold.

4.5 User Interaction / Application Layer

A 7-page Streamlit dashboard (Section 16) lets a user inspect the dataset, the preprocessing strategy, the model comparison and evaluation results, and - on the Live Prediction page - either enter sensor values manually or upload a CSV for batch scoring, with results downloadable as CSV.

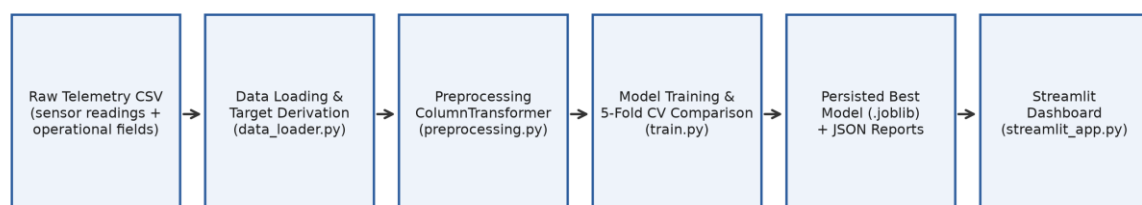


Figure 1. End-to-end system workflow, from raw telemetry CSV to the served prediction, as implemented across `config.py`, `src/data_loader.py`, `src/preprocessing.py`, `src/train.py`, and `streamlit_app.py`.

5. System Architecture

The architecture follows a linear Input → Preprocessing → Model Training/Comparison → Persisted Model → Prediction → Dashboard flow (Figure 1). There is no database, external API, or message broker in the implementation; persistence is entirely file-based (CSV in, Joblib and JSON artefacts out), and the dashboard reads those files directly through cached loader functions.

5.1 Data Layer

A single raw CSV at `data/raw/pump_sensor_data.csv`, read by `src/data_loader.load_raw_data()`. No external database or data warehouse is used; this is consistent with the project being a self-contained demonstrator rather than a connected production system.

5.2 Preprocessing Layer

`src/preprocessing.py` builds a scikit-learn `ColumnTransformer` (Section 7) from the numeric/categorical feature split computed by `identify_feature_types()`. It is embedded inside the same `Pipeline` object as the classifier, so it is always fit and applied together with the model, never as a separate offline step.

5.3 Modelling Layer

`src/train.py` defines the three candidate estimators, cross-validates them, and selects, refits, and persists the best one; `src/evaluate.py` computes point metrics, the confusion matrix, the classification report, ROC curve data, and feature importance/coefficients from a fitted pipeline.

5.4 Serving / Prediction Layer

`src/predict.py` loads the persisted pipeline and exposes `predict_single()` and `predict_batch()`, both built around the input-coercion logic described in Section 18.

5.5 Presentation Layer

`streamlit_app.py` is the only user-facing application. It has no separate backend/API tier - Streamlit executes the Python pipeline functions directly in-process on each interaction, with `@st.cache_data` / `@st.cache_resource` used to avoid recomputation on every rerun.

5.6 Deployment Layer

Not specified in the provided project materials. No Dockerfile, cloud configuration, CI/CD pipeline, or hosting setup was found in the repository; the application is intended to be run locally via streamlit `run streamlit_app.py` (see README.md and Section 15).

6. Dataset Description

The project targets the schema of the public Kaggle “Pump Sensor Data” dataset (timestamp, sensor_00 ... sensor_NN, machine_status), and `src/data_loader.py` / `src/preprocessing.py` are written to work on either that real CSV or a schema-compatible substitute without code changes. The real Kaggle CSV was not present among the materials provided for this case study; the dataset actually used to produce every number, table, and figure in this document is the project's own synthetic generator output (`data/raw/pump_sensor_data.csv`), created by `generate_sample_data.py` and matching the same column schema. This distinction is stated explicitly because the project's own README and generator docstring are explicit that this is a “schema-compatible stand-in”, not the original dataset - results in this report should be read as a demonstration of the pipeline's behaviour on realistic, synthetically generated telemetry, not as validated performance on field data.

The generator simulates a hidden equipment-risk process that ramps from 0 toward 1 ahead of a failure event, triggers a short BROKEN window, and then a longer RECOVERING window before returning to normal. Five of the thirty sensors (sensor_00 – sensor_04) are designated “critical” and drift with this risk signal; the remaining twenty-five are dominated by noise. Sensor cells are additionally dropped at random (missing-completely-at-random) to simulate disconnected/corrupted sensors, and two operational categorical columns (operation_mode, pump_station) are added to exercise the categorical branch of the preprocessing pipeline; the real Kaggle dataset does not include these two columns.

Attribute	Value
Dataset file	<code>data/raw/pump_sensor_data.csv</code> (project-generated synthetic telemetry)
Total records	20,000
Total raw columns	34 (timestamp, 30 sensor readings, 2 categorical, machine_status)
Sampling interval	1-minute cadence, starting 2025-01-01 00:00
Numeric sensor columns	30 (sensor_00 – sensor_29)
Categorical columns	2 (operation_mode, pump_station)
Raw target column	machine_status: NORMAL / RECOVERING / BROKEN
Derived binary target	equipment_failure (0 = Normal, 1 = Failure)
Raw class distribution	NORMAL 19,377 (96.89%); RECOVERING 535 (2.68%); BROKEN 88 (0.44%)
Binary class distribution	Normal 19,377 (96.89%); Failure 623 (3.12%)
Missing sensor cells	23,884 of 600,000 sensor cells (3.98%); 762–837 missing per sensor column
Missing values – other columns	None (timestamp, operation_mode, pump_station, machine_status contain no missing values)
Duplicate rows	0

Table 1. Dataset characteristics of `data/raw/pump_sensor_data.csv`.

6.1 Categorical Feature Detail

Column	Categories (count)
operation_mode	AUTOMATIC 14,958 (74.8%); MANUAL 4,050 (20.3%); MAINTENANCE 992 (5.0%)
pump_station	STATION_A 7,922 (39.6%); STATION_B 7,079 (35.4%); STATION_C 4,999 (25.0%)

Table 2. Category-level breakdown of the two categorical features.

Both categorical columns are operational context rather than part of the original Kaggle schema; they exist so the shared ColumnTransformer's one-hot-encoding branch is genuinely exercised end to end, as noted in the generator's own docstring.

7. Data Preprocessing

All preprocessing is implemented as scikit-learn transformers inside a single ColumnTransformer (src/preprocessing.build_preprocessor), so the same fitted transformation is replayed identically at training, cross-validation, testing, and inference time - it is never a one-off offline script run against the full dataset. Feature columns are split into numeric and categorical purely by pandas dtype (src/preprocessing.identify_feature_types), which is what lets the same code run unchanged on either the real 52-sensor Kaggle CSV (no categoricals) or the enriched synthetic dataset used here.

7.1 Missing-Value Handling

What was done: numeric features (30 sensors + 3 engineered time features) use median imputation (sklearn.impute.SimpleImputer(strategy="median")); categorical features use most-frequent imputation. Why: median imputation is robust to the outlier sensor spikes that occur near failure events, unlike mean imputation, which those spikes would pull away from the typical operating value; most-frequent imputation is the standard robust default for low-cardinality categorical data. How it affects the model: every one of the 20,000 rows remains usable for training rather than being dropped, which matters given that missing values are already concentrated on 23,884 of 600,000 sensor cells - dropping incomplete rows outright would have removed a large share of the (already scarce) failure-related rows.

7.2 Duplicate Removal

What was done: load_raw_data() drops any stray “Unnamed: ...” index columns that CSV exports sometimes introduce. What was found: 0 duplicate rows in the dataset (verified via get_dataset_overview), so no row-level de-duplication logic was required or added.

7.3 Feature Scaling

What was done: StandardScaler (zero mean, unit variance) is applied to all numeric features after imputation. Why: Logistic Regression's coefficient magnitudes and convergence are sensitive to feature scale, and unscaled sensor readings in this dataset span very different ranges (e.g., sensor_00 roughly 48–78 versus sensor_25 roughly 61–78 with a much smaller spread). How it affects the model: it puts all sensors on a comparable scale so that a linear model's coefficients reflect genuine relative influence (Section 13.6) rather than an artefact of a sensor's raw numeric range; tree-based models are scale-invariant but share the same pipeline for simplicity and consistency.

7.4 Categorical Encoding

What was done: OneHotEncoder(handle_unknown="ignore") converts operation_mode and pump_station into binary indicator columns. Why: neither column has a natural ordinal relationship, so one-hot encoding avoids imposing a false ordering that label/ordinal encoding would introduce. The handle_unknown="ignore" setting means an operational category never seen during training degrades to an all-zero encoding at inference time instead of raising an error. How it affects the model: it expands the 2 categorical columns into 6 binary columns (3 operation_mode levels + 3 pump_station levels), all of which enter the model on equal footing with the scaled numeric features.

7.5 Outlier Handling, Class Balancing, and Feature Selection

No separate outlier-removal step (e.g., z-score clipping, IQR filtering) is implemented; outlier sensor spikes are instead implicitly down-weighted by the choice of median (rather than mean) imputation and by StandardScaler, and are, in this dataset, often the genuine failure signal rather than noise to be removed. No resampling technique (e.g., SMOTE, random undersampling) is used; class imbalance is instead addressed at the model level via `class_weight="balanced"` on Logistic Regression and Random Forest (Section 10). No explicit feature-selection step is applied - all 35 engineered features are passed to every candidate model, and the linear model's coefficient magnitudes (Section 13.5) are used descriptively rather than to prune features. Text preprocessing, tokenization, and vectorization are not applicable - the dataset contains no free-text fields.

Branch	Feature count	Missing-value strategy	Transformation
Numeric	33 (30 sensors + <code>hour_of_day</code> , <code>day_of_week</code> , <code>is_weekend</code>)	Median imputation	StandardScaler (zero mean, unit variance)
Categorical	2 (<code>operation_mode</code> , <code>pump_station</code>)	Most-frequent imputation	One-hot encoding (unknown categories ignored at inference)

Table 3. Summary of the ColumnTransformer preprocessing pipeline (src/preprocessing.py).

8. Exploratory Data Analysis

The project's own exploratory analysis is implemented as the Streamlit “Dataset Explorer” page, driven by `src/data_loader.get_dataset_overview()`. It computes dataset shape, column dtypes, missing counts/percentages, duplicate-row count, target class balance, and pandas descriptive statistics (count, mean, std, min, quartiles, max) for numeric columns, and displays a sample of rows. It does not include a feature-correlation matrix, pairwise scatter plots, or automated outlier detection; those are Not specified in the provided project materials and are not claimed here. The two figures below reproduce, from the same underlying computation, the class-balance and missing-value views the dashboard itself renders.

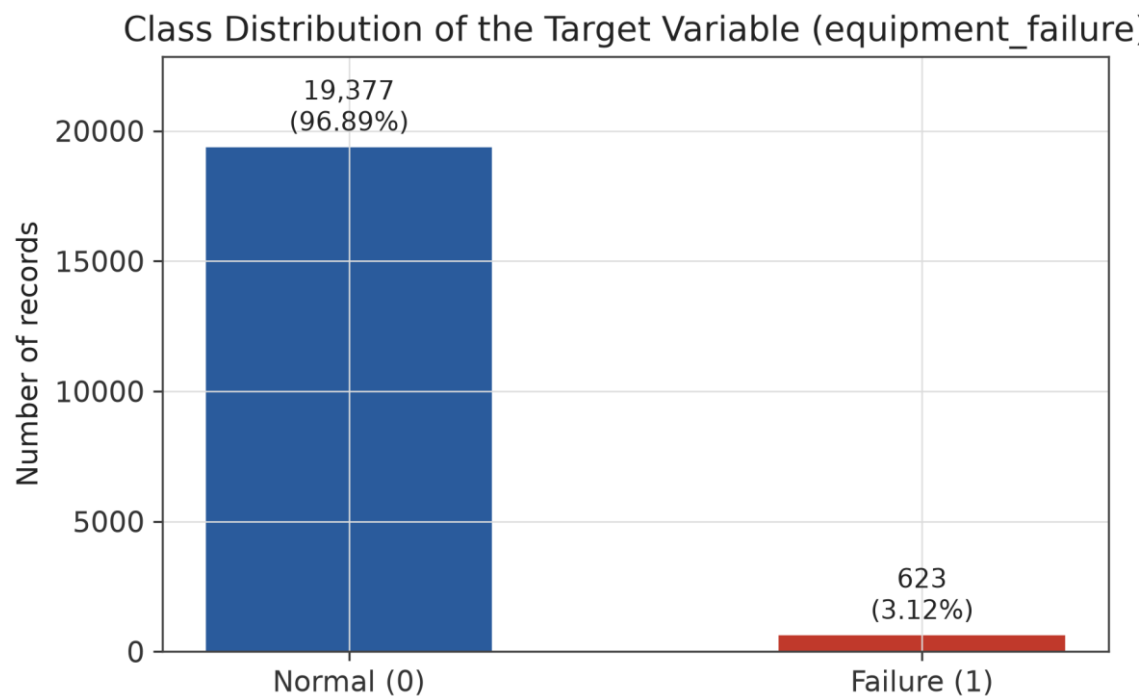


Figure 2. Distribution of the binary target variable equipment_failure across all 20,000 records. The failure class is a small minority (3.12%), motivating the use of class_weight="balanced" and stratified splitting throughout the pipeline.

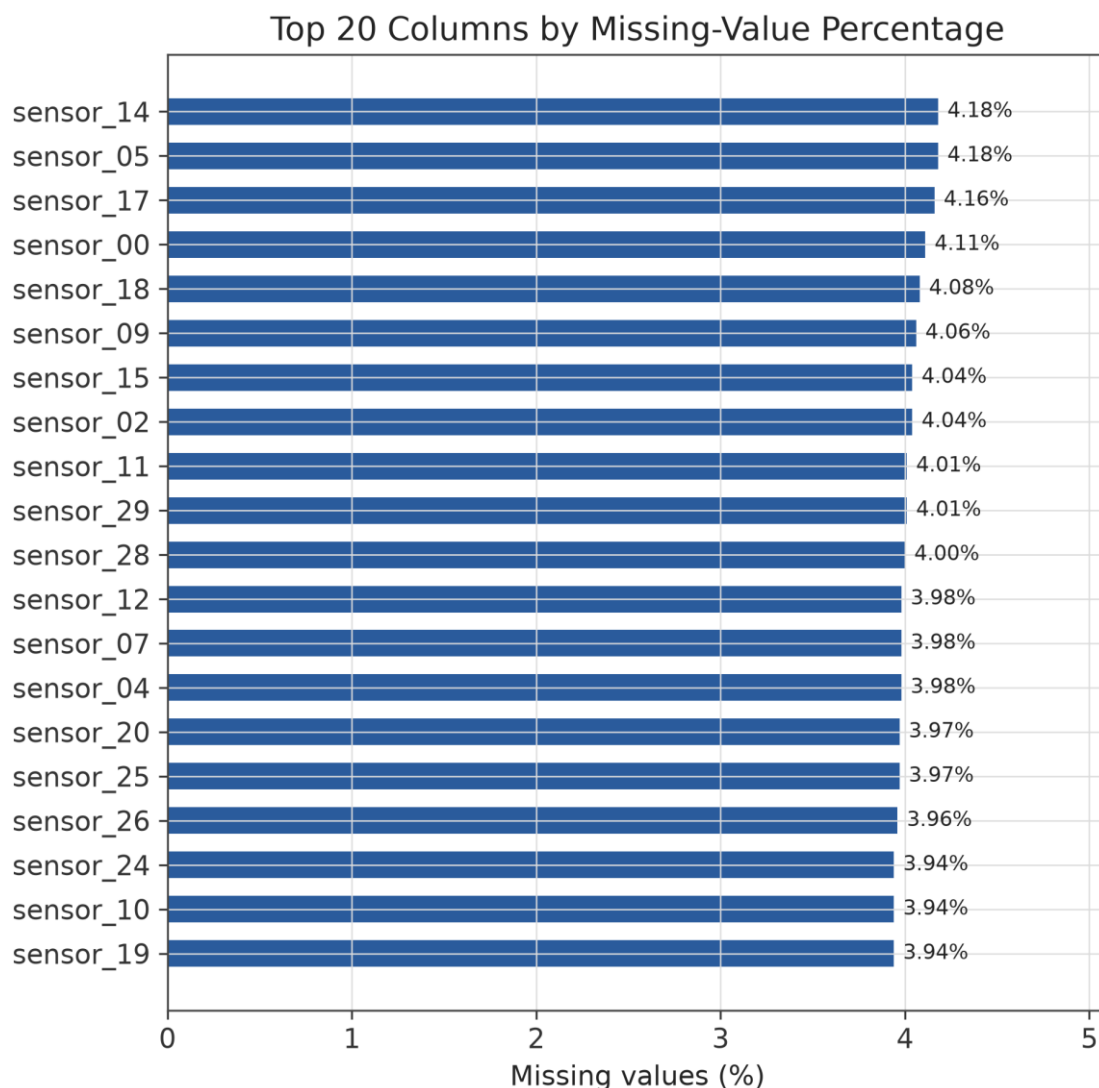


Figure 3. Percentage of missing values for the 20 sensor columns with the highest missingness, matching the Streamlit “Dataset Explorer” page’s missing-values chart. All 30 sensor columns show missingness in a narrow 3.8%-4.2% band (762-837 missing cells each), consistent with the generator’s uniform missing-completely-at-random injection rate; no other column has any missing values.

8.1 Descriptive Statistics for Representative Sensors

Table 4 reports descriptive statistics for four representative sensors: `sensor_00` and `sensor_02` are among the five “critical” sensors engineered to drift with the underlying failure-risk process, while `sensor_15` and `sensor_25` are non-critical (predominantly noise) channels. Note the visibly wider spread (std) and range on the critical sensors relative to a low-variance channel like `sensor_25` - a pattern consistent with the drift-plus-noise structure documented in `generate_sample_data.py`, and one that the trained model’s coefficients (Section 13.6) end up reflecting.

Sensor	Count	Mean	Std Dev	Min	25%	50%	75%	Max
sensor_00 (critical)	19,178	58.75	3.61	48.54	56.50	58.24	60.24	78.44
sensor_02 (critical)	19,193	42.15	3.23	35.22	40.36	41.48	42.78	59.98
sensor_15 (non-critical)	19,193	48.13	5.72	25.98	44.33	48.15	52.01	69.42
sensor_25 (non-critical)	19,206	69.93	1.90	61.46	68.67	69.92	71.21	77.58

Table 4. Descriptive statistics for four representative sensor columns (values as reported by `pandas.DataFrame.describe()`).

9. Feature Engineering and Feature Extraction

Feature engineering is limited to a small, well-justified set of transformations rather than an extensive hand-crafted feature set - appropriate given that the 30 sensor channels already carry the primary predictive signal by construction.

9.1 Time-Feature Engineering

`src/data_loader.engineer_time_features()` replaces the raw timestamp column with three derived features: `hour_of_day` (0–23), `day_of_week` (0=Monday ... 6=Sunday), and `is_weekend` (binary). The raw timestamp itself is excluded from the model inputs (`config.NON_FEATURE_COLUMNS`) because, as a near-unique identifier, it would not generalise to new timestamps; hour-of-day and day-of-week, by contrast, can capture real shift-pattern or duty-cycle effects on equipment stress.

9.2 Target Derivation

`src/data_loader.derive_target()` maps the three-level `machine_status` column to the binary `equipment_failure` target using `config.STATUS_TO_FAILURE_MAP` (NORMAL → 0; RECOVERING → 1; BROKEN → 1). If a dataset already ships a binary `equipment_failure` column, it is validated and used as-is instead.

9.3 Feature Selection and Encoding

No dimensionality-reduction or automated feature-selection step (e.g., PCA, recursive feature elimination) is applied; all 33 numeric and 2 categorical features are retained. Encoding is handled entirely by the shared preprocessing `ColumnTransformer` described in Section 7 - one-hot encoding for the 2 categorical columns (expanding to 6 indicator columns) and scaling for the 33 numeric columns, for 39 total transformed input dimensions to the classifier.

Group	Features	Count
Raw sensor readings	sensor_00 ... sensor_29	30
Engineered time features	hour_of_day, day_of_week, is_weekend	3
Operational categorical features	operation_mode, pump_station	2
Excluded (non-feature) columns	timestamp, machine_status, equipment_failure, row_id	n/a

Table 5. Feature groups used as model input (35 raw feature columns; 39 after one-hot expansion).

10. Machine Learning Methodology

Three supervised binary classifiers are implemented and compared in `src/train.get_candidate_models()`, each wrapped with the same preprocessing `ColumnTransformer` inside a single scikit-learn Pipeline so that preprocessing and modelling are always fit and evaluated together.

10.1 Logistic Regression

1. What it is: a linear model that estimates the probability of the positive class (Failure) via the logistic (sigmoid) function applied to a weighted sum of the input features.
2. How it works conceptually: it learns one coefficient per (transformed) feature; a positive coefficient increases predicted failure probability as that feature rises, a negative coefficient decreases it. Training minimises log loss via the `lbfgs` solver (scikit-learn's default).
3. Why it was selected: as the simplest, most interpretable of the three candidates, it serves as a well-understood baseline and - per `config.STATUS_TO_FAILURE_MAP`-derived class imbalance - supports a built-in class-weighting mechanism to counter the 3.12% minority failure rate.
4. How it was applied: fit inside the shared Pipeline (`ColumnTransformer` → `LogisticRegression`) using scikit-learn's `LogisticRegression` class, compared via 5-fold CV, and ultimately selected as the best model by mean CV F1-score (Section 13).
5. Important hyperparameters used: `max_iter=1000` (raised from the library default of 100 to ensure convergence on 39 input dimensions); `class_weight="balanced"` (inversely weights each class by its frequency, so misclassifying the rare Failure class is penalised roughly $31\times$ more than misclassifying Normal); `random_state=42` for reproducibility; solver left at its scikit-learn default (`lbfgs`).
6. Advantages: fast to train (3.94 s mean CV fit time - the fastest of the three), coefficients are directly interpretable (Section 13.6), and `class_weight="balanced"` gives it strong recall on the minority class without any resampling step.
7. Limitations: assumes an approximately linear decision boundary in the (scaled) feature space, so it cannot natively capture complex feature interactions the way the tree ensembles can; in this project it also produced the lowest precision of the three candidates (Section 13).

10.2 Random Forest

1. What it is: an ensemble of decision trees, each trained on a bootstrap-resampled subset of the training data and a random subset of features at each split, with predictions aggregated by majority vote / averaged class probability.
2. How it works conceptually: individual trees overfit their own bootstrap sample, but averaging many de-correlated trees reduces variance and typically improves generalisation over a single tree.
3. Why it was selected: it is a strong, low-tuning-effort baseline for tabular sensor data and, like Logistic Regression, supports `class_weight="balanced"` for imbalance handling.
4. How it was applied: `RandomForestClassifier` inside the same shared Pipeline, compared under the identical 5-fold CV protocol as the other two candidates.

5. Important hyperparameters used: `n_estimators=300` (number of trees); `max_depth=None` (trees grown until leaves are pure or reach the minimum-samples-split default, i.e., no explicit depth cap); `class_weight="balanced"`; `random_state=42`; `n_jobs=-1` (parallel tree building).

6. Advantages: captures non-linear feature interactions, provides a built-in `feature_importances_` measure (Section 13.6 discusses why this is not the measure ultimately shown, since Random Forest was not the selected model), and is generally robust to unscaled or noisy features.

7. Limitations: in this project's results, its mean CV recall on the Failure class (26.70%) was far below Logistic Regression's (81.11%) despite also using `class_weight="balanced"` - an ensemble of many trees each still tends to predict the majority class at the individual-tree level unless the imbalance is addressed more aggressively (e.g., a lower decision threshold or explicit resampling); it is also more expensive to store and slower to fit (9.68 s mean) than Logistic Regression.

10.3 Gradient Boosting

1. What it is: an ensemble method that builds decision trees sequentially, where each new tree is fit to the residual errors (via gradient descent on the loss function) of the ensemble built so far.

2. How it works conceptually: rather than averaging independent trees (as Random Forest does), it incrementally corrects the current model's mistakes, which often yields higher predictive accuracy at the cost of longer training time and more sensitivity to hyperparameters such as learning rate.

3. Why it was selected: to compare a boosting-based ensemble against the bagging-based Random Forest and the linear Logistic Regression baseline under the same protocol.

4. How it was applied: `GradientBoostingClassifier` inside the shared Pipeline, evaluated identically to the other two candidates via 5-fold CV.

5. Important hyperparameters used: `n_estimators=200` (boosting stages); `learning_rate=0.1`; `max_depth=3` (shallow trees, typical for boosting); `random_state=42`. Unlike the other two candidates, scikit-learn's `GradientBoostingClassifier` exposes no `class_weight` parameter, so this model relies entirely on the CV comparison to reveal whether the resulting class imbalance sensitivity is a problem - which, per Section 13, it was: it achieved the best CV ROC-AUC (0.9490) and precision (0.5775) but the second-lowest recall (0.2549).

6. Advantages: achieved the highest mean CV accuracy (97.11%), precision (57.75%), and ROC-AUC (0.9490) of the three candidates, and generally offers strong predictive performance on structured/tabular data.

7. Limitations: by far the slowest to train (34.73 s mean CV fit time - roughly 9× Logistic Regression and 3.6× Random Forest); no built-in class-weighting mechanism, contributing to its comparatively low recall on the rare failure class; more hyperparameters to tune than the other two candidates, none of which were tuned in this project (Section 11).

10.4 Candidate Model Configuration Summary

Model	Key hyperparameters (as coded in src/train.py)
Logistic Regression	max_iter=1000, class_weight="balanced", random_state=42, solver=lbfgs (default)
Random Forest	n_estimators=300, max_depth=None, class_weight="balanced", random_state=42, n_jobs=-1
Gradient Boosting	n_estimators=200, learning_rate=0.1, max_depth=3, random_state=42

Table 6. Hyperparameters used for each candidate classifier. No automated search (e.g., GridSearchCV, RandomizedSearchCV) was performed; these are fixed values chosen for a first comparison pass.

11. Model Training

The full training procedure is implemented in `src/train.run_training_pipeline()` and is run from the command line via `python -m src.train` (or the dashboard's "Train / Retrain Models" button, which calls the same function).

11.1 Train / Test Split

The dataset is split once, up front, into an 80% training portion and a 20% held-out test portion using scikit-learn's `train_test_split` with `stratify=y`, so the 3.12% failure rate is preserved in both splits. This split is performed before any cross-validation or model fitting, and the test portion is touched exactly once - for the final evaluation in Section 13 - never during model selection.

11.2 Validation Strategy

Model comparison uses `StratifiedKFold(n_splits=5, shuffle=True, random_state=42)` via scikit-learn's `cross_validate`, scored simultaneously on accuracy, precision, recall, F1-score, and ROC-AUC (`config.CV_SCORING`). Because the `ColumnTransformer` is embedded inside the cross-validated Pipeline, it is refit independently on each of the 5 training folds - imputation medians, scaling statistics, and one-hot categories are computed only from that fold's training rows, so no information from a validation fold (or from the held-out test split) ever leaks into preprocessing statistics.

11.3 Hyperparameter Tuning

No automated hyperparameter search was performed in this implementation. Each candidate model uses the fixed hyperparameters listed in Table 6; the 5-fold CV step is used purely for model selection among the three candidates, not for tuning any individual model's hyperparameters.

11.4 Class Balancing and Feature Scaling

Class imbalance is handled via `class_weight="balanced"` on Logistic Regression and Random Forest (Section 10); no resampling (e.g., SMOTE) is used. Feature scaling (`StandardScaler`) and categorical encoding are applied identically across all three candidates through the shared `ColumnTransformer`, so differences in Section 13's results reflect the algorithms themselves rather than differing preprocessing.

11.5 Training Procedure and Artefacts

1. Load and prepare the dataset (`load_raw_data` → `engineer_time_features` → `derive_target`).
2. Identify numeric/categorical features and build the shared `ColumnTransformer`.
3. Stratified 80/20 train/test split (`random_state=42`).
4. 5-fold stratified cross-validation of all three candidates on the training split only.
5. Select the candidate with the highest mean CV F1-score (`config.SELECTION_METRIC = "f1"`).
6. Refit the selected candidate's full Pipeline on the entire training split (16,000 rows).
7. Evaluate the refit pipeline once on the untouched 4,000-row test split.

8. Persist the fitted pipeline (models/best_model.joblib), the CV comparison table (models/model_comparison_results.json), the test evaluation report (models/evaluation_report.json), and feature/run metadata (models/feature_metadata.json).

Every value in this document's tables and figures is read from these four persisted artefacts, generated in the project's environment on 2026-08-05 (05:34:25 UTC, per feature_metadata.json), using Python 3.14, scikit-learn 1.9.0, pandas 3.0.5, and joblib 1.5.3 (Section 15.1).

Split	Rows	Failure-class rows	Failure rate
Full dataset	20,000	623	3.12%
Training split (80%)	16,000	498	3.11% (stratified)
Test split (20%, held out)	4,000	125	3.13% (stratified)

Table 7. Stratified train/test split summary (random_state=42, test_size=0.2).

12. Model Evaluation Methodology

Evaluation is implemented in `src/evaluate.py` and applied identically during cross-validation (Section 11) and to the final held-out test evaluation (Section 13). All metrics below are computed with `zero_division=0`, which returns 0 rather than raising or emitting a warning on the rare CV fold where a class is entirely absent from a batch of predictions.

12.1 Point Metrics

Accuracy - the proportion of all predictions that are correct. Reported for completeness but, given the 3.12% failure rate, a poor standalone indicator of failure-detection quality (a model that always predicts “Normal” would score 96.9% accuracy while catching zero failures).

Precision - of all records the model predicted as Failure, the proportion that were actually a failure. Low precision means frequent false alarms.

Recall - of all records that were actually a failure, the proportion the model correctly flagged. Low recall means missed failures, which in a predictive-maintenance context is typically the more costly error.

F1-score - the harmonic mean of precision and recall, used in this project as the single model-selection metric (`config.SELECTION_METRIC`) because it balances both failure modes rather than favouring one at the total expense of the other.

ROC-AUC - the area under the Receiver Operating Characteristic curve (true-positive rate vs. false-positive rate across all decision thresholds), summarising ranking quality independently of the specific 0.5 decision threshold used elsewhere.

12.2 Confusion Matrix

A 2×2 table of actual vs. predicted class counts (true negatives, false positives, false negatives, true positives), computed via `sklearn.metrics.confusion_matrix`. It is the most direct way to see the precision/recall trade-off in absolute terms - how many real failures were caught, missed, or how many false alarms were raised - rather than as a single summary statistic.

12.3 Classification Report

Per-class precision, recall, F1-score, and support (row count), plus macro- and weighted-average rollups, computed via `sklearn.metrics.classification_report`. The macro average treats both classes equally regardless of size; the weighted average is dominated by the majority Normal class.

12.4 ROC Curve

The full false-positive-rate/true-positive-rate curve (`sklearn.metrics.roc_curve`) underlying the single ROC-AUC number, useful for visually assessing whether the model's ranking of failure probability is informative across the whole threshold range, not only at the default 0.5 cutoff.

13. Results

13.1 Cross-Validation Comparison (Training Split)

Table 8 and Figure 4 report the stratified 5-fold cross-validation results (mean $\pm$ standard deviation across the 5 folds) for all three candidates, computed on the 16,000-row training split only, exactly as persisted in `models/model_comparison_results.json`.

Model	Accuracy	Precision	Recall	F1-Score	ROC-AUC	Fit time (s)
Logistic Regression	0.9176 $\pm$ 0.0018	0.2480 $\pm$ 0.0102	0.8111 $\pm$ 0.0459	0.3798 $\pm$ 0.0167	0.9252 $\pm$ 0.0113	3.94
Gradient Boosting	0.9711 $\pm$ 0.0016	0.5775 $\pm$ 0.0533	0.2549 $\pm$ 0.0403	0.3532 $\pm$ 0.0475	0.9490 $\pm$ 0.0115	34.73
Random Forest	0.9660 $\pm$ 0.0017	0.4292 $\pm$ 0.0400	0.2670 $\pm$ 0.0156	0.3285 $\pm$ 0.0189	0.9388 $\pm$ 0.0130	9.68

Table 8. Stratified 5-fold cross-validation results, sorted by mean F1-score (the model-selection metric).
Source: `models/model_comparison_results.json`.

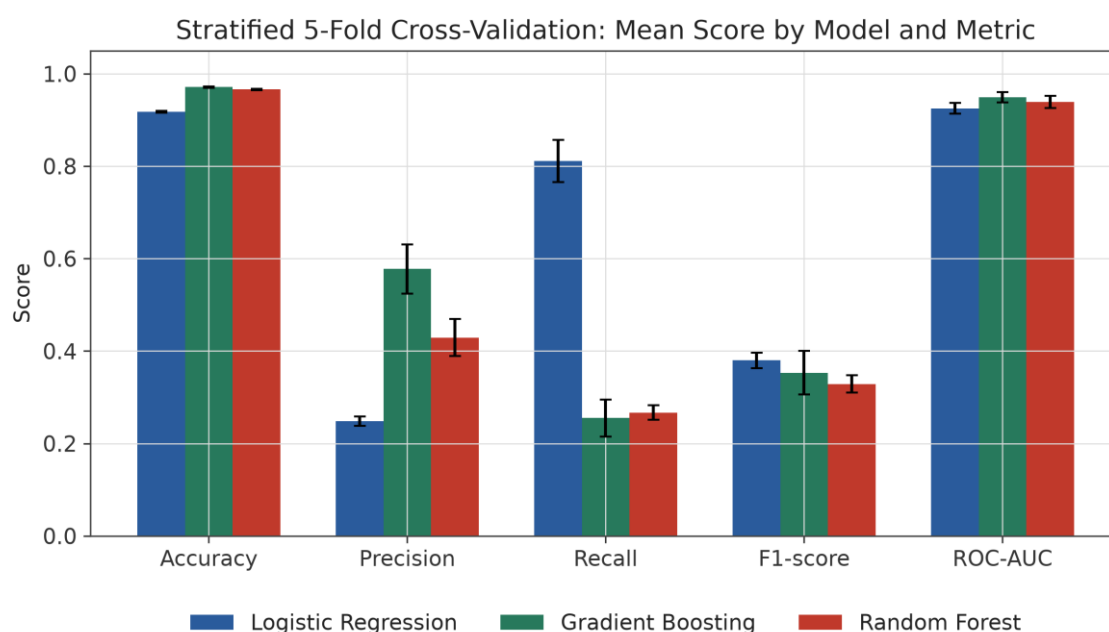


Figure 4. Mean cross-validated score (± 1 standard deviation) for each of the three candidate models across all five scoring metrics.

By the project's selection rule (highest mean CV F1-score), Logistic Regression is chosen as the best model (0.3798, versus 0.3532 for Gradient Boosting and 0.3285 for Random Forest), driven primarily by its substantially higher recall (81.11%) - matching `feature_metadata.json`'s recorded `best_model_name`: "Logistic Regression".

13.2 Held-Out Test Evaluation (Selected Model: Logistic Regression)

The selected model is refit once on the full 16,000-row training split and evaluated once on the 4,000-row held-out test split, which was not used in cross-validation or model selection. Results are read directly from `models/evaluation_report.json`.

Metric	Value
Accuracy	0.9195 (91.95%)
Precision (Failure class)	0.2568 (25.68%)
Recall (Failure class)	0.8320 (83.20%)
F1-score (Failure class)	0.3925
ROC-AUC	0.9395

Table 9. Held-out test-set metrics for the selected Logistic Regression pipeline ($n = 4,000$).

13.3 Confusion Matrix

Confusion Matrix – Logistic Regression (Test Set, $n=4000$)

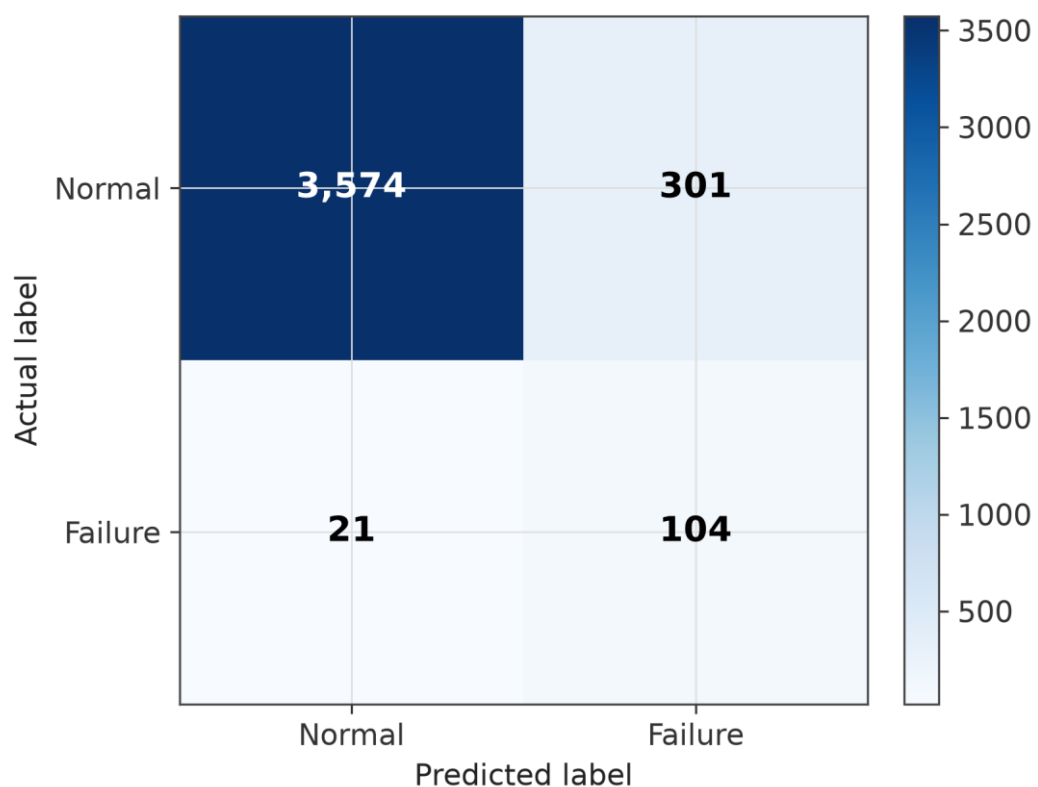


Figure 5. Confusion matrix for the Logistic Regression pipeline on the 4,000-row held-out test set.

Of 125 genuinely failing/recovering records in the test set, the model correctly flagged 104 (true positives) and missed 21 (false negatives) - a recall of 83.2%. Of the 3,875 genuinely normal records, 3,574 were correctly identified and 301 were incorrectly flagged as a failure (false positives), which is

what drives precision down to 25.68%: roughly 3 out of every 4 “Failure” alerts the model raises on this test set are false alarms.

13.4 ROC Curve

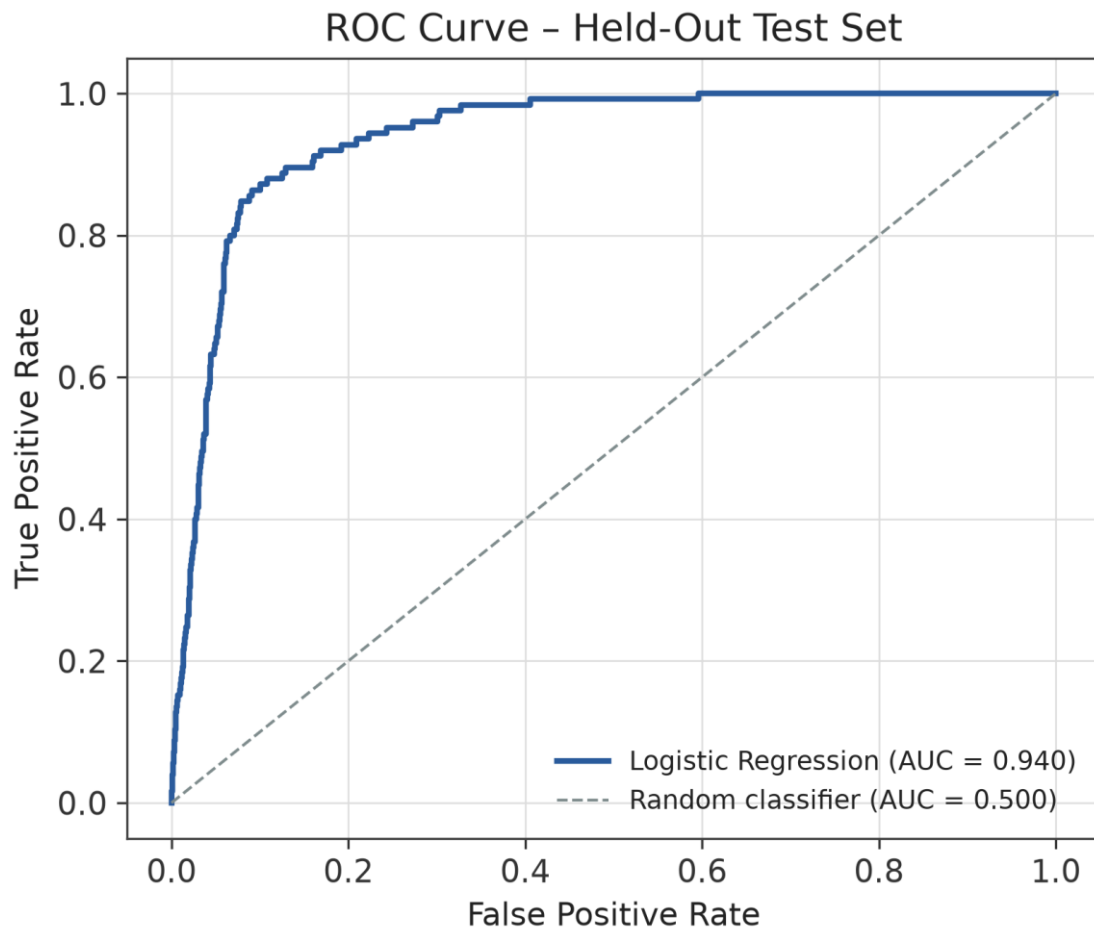


Figure 6. Receiver Operating Characteristic curve for the Logistic Regression pipeline on the held-out test set (AUC = 0.940).

An ROC-AUC of 0.9395 indicates that, independent of the specific 0.5 decision threshold used elsewhere in this report, the model ranks a randomly chosen failure record above a randomly chosen normal record about 94% of the time - i.e., the underlying probability estimates are considerably more informative than the precision figure alone suggests; the low precision is a consequence of the fixed 0.5 threshold choice (Section 19), not of poor ranking quality.

13.5 Classification Report

Class	Precision	Recall	F1-Score	Support
Normal	0.9942	0.9223	0.9569	3,875
Failure	0.2568	0.8320	0.3925	125
Accuracy			0.9195	4,000
Macro avg	0.6255	0.8772	0.6747	4,000
Weighted avg	0.9711	0.9195	0.9393	4,000

Table 10. Full per-class classification report for the held-out test set. Source: `models/evaluation_report.json`.

13.6 Feature Importance (Selected Model: Logistic Regression Coefficients)

Because the selected model is linear, it has no `feature_importances_` attribute; instead, absolute coefficient magnitude on the standardised feature space is used as the influence measure, exactly as the dashboard's own Feature Importance page does for a linear model. Coefficients were extracted directly from the persisted `models/best_model.joblib` pipeline.

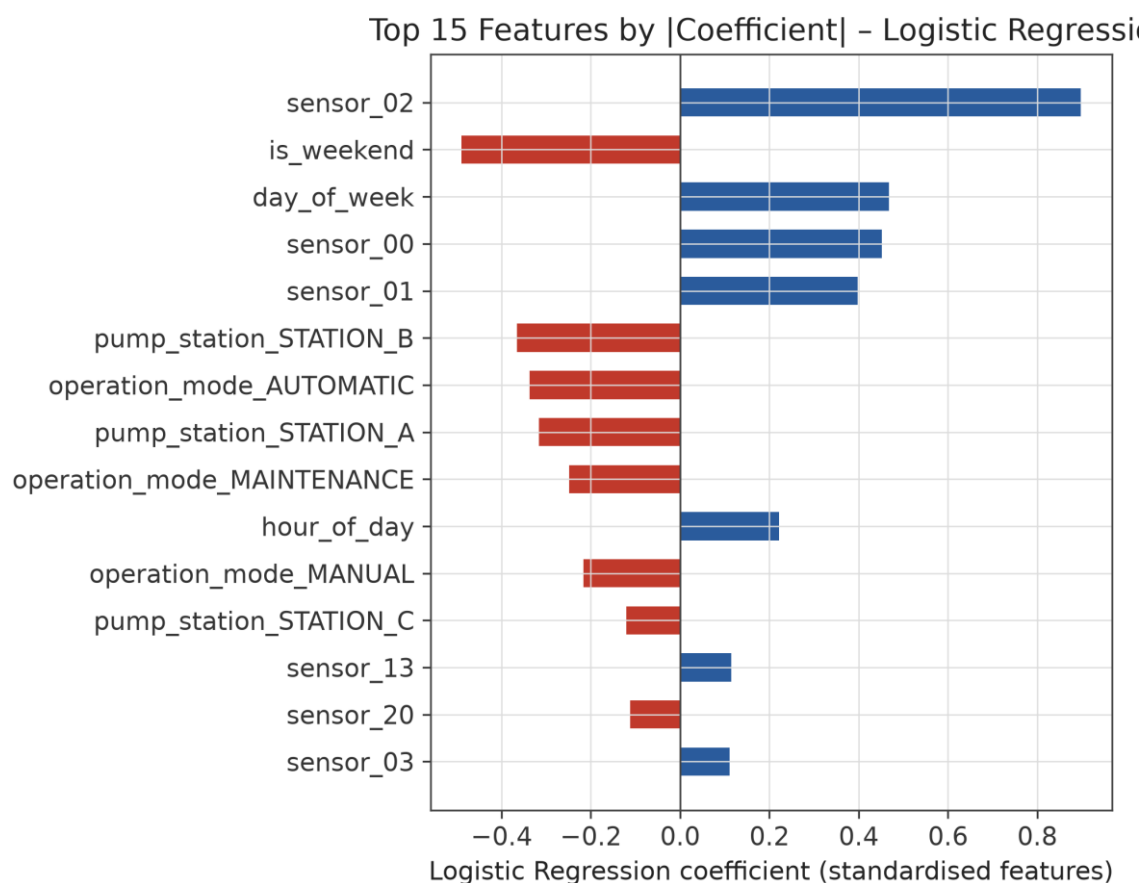


Figure 7. Top 15 features by absolute Logistic Regression coefficient magnitude. Positive bars (blue) push the prediction toward Failure as the (standardised) feature increases; negative bars (red) push toward Normal.

The two largest-magnitude sensor coefficients belong to `sensor_02` (+0.898) and `sensor_00` (+0.452), followed closely by `sensor_01` (+0.397); `sensor_13`, `sensor_20`, and `sensor_03` also appear in the top 15. This is a meaningful internal consistency check: the synthetic data generator (Section 6)

designates sensor_00 through sensor_04 as the five “critical” sensors that drift proportionally with the hidden failure-risk signal, while the remaining 25 sensors are predominantly noise. Four of the model's most influential sensor features are drawn from exactly that critical set, indicating the model is picking up genuine engineered signal rather than fitting noise. The remaining top features are operational context - is_weekend, day_of_week, hour_of_day, and the one-hot pump_station / operation_mode indicators - showing the model also leans on operating-context information rather than sensor readings alone.

14. Model Comparison and Selection

Comparing Table 8's cross-validation results, the three candidates split cleanly into two behaviours. Logistic Regression trades accuracy and precision for a much higher recall: it is 3–5 percentage points less accurate than the tree ensembles, but catches roughly three times as many real failures (81.1% CV recall vs. 25.5%–26.7%). Random Forest and Gradient Boosting sit close together on accuracy (96.6%–97.1%) and both trail badly on recall, despite Random Forest also using `class_weight="balanced"`; Gradient Boosting additionally has no class-weighting mechanism at all and is the most expensive to train by a wide margin (34.73 s vs. 3.94 s and 9.68 s).

Because `config.SELECTION_METRIC` is F1-score - a balance of precision and recall - and because a missed failure is generally more costly than a false alarm in predictive maintenance, Logistic Regression is the preferred model on the project's own selection criterion, and this is the model whose test-set results are reported in Section 13.2–13.6. This preference is defensible for this use case: catching 83% of real failures at the cost of a high false-alarm rate is generally more useful to a maintenance team than catching only 25–27% of failures with fewer false alarms, provided the false-alarm rate is manageable downstream (Section 19 discusses why it currently may not be).

A genuinely open question the results raise - and one this project does not resolve - is why Random Forest's recall stayed low (26.70%) despite also using `class_weight="balanced"`. This suggests that, for this dataset, threshold adjustment or explicit resampling (Section 20) may be more effective levers for the tree-based models than class weighting alone.

From a computational standpoint, Logistic Regression is also the cheapest of the three to train and score, which is a practical advantage for the “Train / Retrain Models” button exposed on the dashboard (Section 16), where a user can trigger retraining interactively and expects a response within seconds.

15. System Implementation

The system is implemented entirely in Python, structured as a small package (`src/`) of pure functions with no side effects beyond explicit file I/O, a single configuration module (`config.py`) as the source of truth for paths/schema/hyperparameters, and a Streamlit script (`streamlit_app.py`) as the only user-facing entry point besides the two command-line scripts (`generate_sample_data.py`, `python -m src.train`).

15.1 Technology Stack

Category	Technology
Language	Python 3.14
ML / preprocessing	scikit-learn 1.9.0 (ColumnTransformer, Pipeline, LogisticRegression, RandomForestClassifier, GradientBoostingClassifier)
Data handling	pandas 3.0.5, NumPy 2.5.1
Model persistence	joblib 1.5.3
Interactive visualization (dashboard)	Plotly 6.9.0
Web application / dashboard	Streamlit 1.61.0
Report figures (this document)	Matplotlib, rendered from the project's stored JSON results and persisted model

Table 11. Technology stack, as declared in `requirements.txt` and as installed in the project's local virtual environment (`venv/`) at the time of this review.

`requirements.txt` itself pins minimum versions only (`pandas>=2.0`, `numpy>=1.24`, `scikit-learn>=1.3`, `joblib>=1.3`, `streamlit>=1.30`, `plotly>=5.18`); the specific versions in Table 11 are what is actually installed in the project's `venv/` directory. There is no database, REST API layer, authentication, or cloud deployment configuration in the project; all persistence is local files (CSV, Joblib, JSON), and the application is run locally via `streamlit run streamlit_app.py`, per `README.md`.

16. Application Workflow

End-to-end, the application workflow is: User Input (manual sensor form or batch CSV upload) → Schema Coercion (missing/unparseable/infinite values mapped to NaN) → the fitted ColumnTransformer's `.transform()` (impute + scale/encode, replaying the exact statistics learned during training) → the loaded Pipeline's `.predict_proba()` → a failure probability and Normal/Failure label rendered in the dashboard.

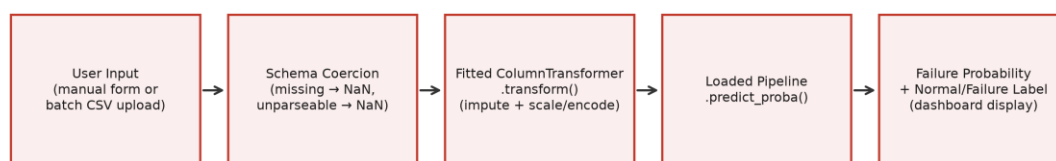


Figure 8. Prediction-time workflow implemented in `src/predict.py`, from raw user input to the displayed failure probability.

This workflow is deliberately identical for both the “Single Reading” and “Batch CSV Upload” modes on the Live Prediction dashboard page: `predict_single()` and `predict_batch()` share the same column-coercion helpers (`_coerce_input_row` / `_coerce_numeric`), so a missing sensor value behaves the same way - imputed away, not rejected - whether it arrives as one manually entered form field or as one blank cell inside an uploaded CSV of thousands of rows.

16.1 Dashboard Pages

Page	What it shows
Overview	Headline stats, pipeline summary, current best model
Dataset Explorer	Shape, missing values, class balance, descriptive stats
Preprocessing	Numeric/categorical split, imputation & encoding strategy
Model Comparison	Stratified k-fold CV results across 3 models, 5 metrics
Evaluation	Held-out test metrics, confusion matrix, ROC curve, classification report
Feature Importance	Tree-model feature importances (or coefficients for linear models)
Live Prediction	Manual sensor-value entry, or batch CSV scoring, with failure probability

Table 12. The 7 pages of the Streamlit dashboard (`streamlit_app.py`), as documented in `README.md` and confirmed against the `PAGES` dictionary in `streamlit_app.py`.

No screenshots of the running dashboard were included among the project materials provided for this case study, so this section describes the interface from the application's source code (`streamlit_app.py`) rather than reproducing captured images; the figures throughout this document (class balance, missing values, model comparison, confusion matrix, ROC curve, feature importance)

are re-renderings, from the same underlying stored results, of the charts each dashboard page displays interactively via Plotly.

16.2 Resilience to Malformed Input

Per the original project brief (Section 1.1), the Live Prediction page is designed not to crash on imperfect input. Missing form fields or CSV columns become NaN and are imputed by the fitted pipeline exactly as during training; values in numeric columns that are not parseable as numbers, or that are infinite, are also coerced to NaN rather than raised as an error (`src/predict._coerce_numeric`); unrecognised categorical values degrade to an all-zero one-hot encoding (`OneHotEncoder(handle_unknown="ignore")`); extra, unexpected CSV columns are silently dropped. Errors that genuinely cannot be recovered from (e.g., a completely empty input dictionary) raise a specific `PredictionInputError` that the dashboard catches and displays as a readable message rather than a stack trace.

17. Key Findings

- The dataset is a strongly imbalanced binary classification problem - only 3.12% of records (623 of 20,000) are failure-related - a realistic reflection of how rare true failure events are relative to normal operating time in industrial settings.
- Despite being the simplest of the three candidates, Logistic Regression achieved the best mean cross-validated F1-score (0.3798) and by far the best recall (81.11% CV / 83.20% test), because `class_weight="balanced"` strongly penalises missed failures - the error type that matters most in a predictive-maintenance setting.
- The tree ensembles achieved much higher accuracy (96.6%–97.1%) and precision (43%–58%) but far lower recall (25%–27%); Random Forest's low recall despite also using `class_weight="balanced"` is a notable, unresolved finding worth further investigation (Section 20).
- The selected model's coefficients concentrate on `sensor_00`–`sensor_03` and `sensor_13` - largely overlapping with the five sensors (`sensor_00`–`sensor_04`) the synthetic generator was designed to make “critical” drifting indicators of the underlying failure-risk process - a meaningful internal consistency check that the model is learning genuine signal rather than noise (Section 13.6).
- An ROC-AUC of 0.9395 shows the model's underlying probability ranking is considerably stronger than its precision figure alone suggests; the low precision (25.68%) is a consequence of using a fixed 0.5 decision threshold on a heavily imbalanced problem, not evidence of poor ranking ability.
- The resilience requirements from the original project brief (Section 1.1) were validated against concrete bugs, not only claimed: two dedicated commits (“Fix batch/single predictions failing entirely on partial sensor corruption” and “Coerce infinite sensor readings to NaN instead of failing the whole batch”) show the input-coercion logic in `src/predict.py` was iteratively hardened after real failure modes were found (Section 18).

18. Challenges Encountered

18.1 Severe Class Imbalance

With only 3.12% of records labelled Failure, a model optimised naively for accuracy would learn to almost always predict Normal. This was addressed with `class_weight="balanced"` on two of the three candidates and stratified sampling in both the train/test split and the cross-validation folds, but Table 8 shows the challenge was only partly overcome - precision on the Failure class remains low for every candidate (25.68%–57.75%), and Random Forest's recall stayed low despite class weighting.

18.2 Missing and Corrupted Sensor Data

23,884 of 600,000 sensor cells (3.98%) are missing in the working dataset, simulating disconnected or unreadable sensors. This was addressed structurally through the fitted median/most-frequent imputers in the shared ColumnTransformer, which are trained once and then reused identically at inference time.

18.3 Prediction-Time Input Robustness

The project's git history shows this was not solved correctly on the first attempt: two later commits - “Fix batch/single predictions failing entirely on partial sensor corruption” and “Coerce infinite sensor readings to NaN instead of failing the whole batch” - indicate that an earlier version of `src/predict.py` could be made to fail on realistic bad input (e.g., a single corrupted or infinite sensor reading crashing an entire batch prediction), directly contradicting the original project brief's “without runtime errors” requirement. This was resolved by making all numeric coercion explicit and schema-driven (`_coerce_numeric`, `_expected_numeric_columns`) rather than relying on pandas' default type inference.

18.4 No Access to the Original Kaggle Dataset

The real Kaggle “Pump Sensor Data” CSV was not available among the materials provided for this project/case study. This was addressed by building a schema-compatible synthetic generator (`generate_sample_data.py`) so the full pipeline could still be developed, trained, and demonstrated end to end - at the cost of the results in this report reflecting synthetic rather than field data (Section 6, Section 19).

19. Limitations

- Synthetic training data: every result in this report was produced on the project's own generated telemetry, not the real Kaggle “Pump Sensor Data” dataset the schema is modelled on; performance on real plant sensor data is unverified and may differ materially.
- Low precision on the selected model: 25.68% precision on the test set means roughly 3 out of 4 “Failure” alerts are false alarms at the default 0.5 threshold, which could cause alert fatigue if deployed as-is without threshold tuning or a downstream triage step.
- No hyperparameter tuning: all three candidates use fixed, manually chosen hyperparameters (Section 10.4); no grid/random/Bayesian search was performed, so reported performance is not necessarily each algorithm's best achievable performance on this data.
- Single train/test split and single random seed: results are reported from one 80/20 split and one `random_state=42` run; variability of test-set performance across different splits/seeds was not assessed beyond the 5-fold CV standard deviations already reported in Table 8.
- No temporal validation: the train/test split is a random stratified split, not a chronological split; it does not test the model's ability to generalise forward in time to failure events after the training period, which is the more realistic evaluation for a deployed predictive-maintenance system.
- No production deployment or monitoring: the system runs locally via streamlit run; there is no containerization, cloud hosting, authentication, logging/alerting integration, or model-drift monitoring in the current implementation.
- Limited real-world testing: the dashboard and prediction module have been exercised against the synthetic dataset and ad hoc malformed inputs (Section 18.3) but not against a live sensor feed or field-deployed hardware.

20. Future Enhancements

The following are proposed improvements, not implemented features; they are listed to distinguish clearly between what the current system does (Sections 4–16) and what could reasonably extend it.

1. Validate the pipeline on the real Kaggle “Pump Sensor Data” dataset (or genuine field telemetry) once available, since every result in this report currently reflects synthetic data only.
2. Perform systematic hyperparameter optimisation (e.g., GridSearchCV, RandomizedSearchCV, or Bayesian optimisation) for each candidate model instead of the fixed configuration in Section 10.4.
3. Tune the decision threshold (currently a fixed 0.5) or apply cost-sensitive learning to better balance the false-alarm rate against missed-failure rate for a specific deployment's tolerance.
4. Evaluate additional model families, such as gradient-boosted tree libraries (XGBoost, LightGBM) or sequence models (e.g., LSTM/temporal convolutional networks) that can exploit the time-ordered nature of telemetry more directly than the current per-row classifiers.
5. Investigate explicit class-imbalance techniques beyond `class_weight`, such as SMOTE or undersampling, particularly to address Random Forest's comparatively low recall despite class-weighting (Section 14).
6. Add model explainability tooling (e.g., SHAP) for the tree-based candidates, extending the coefficient-based interpretation currently available only for the linear model (Section 13.6).
7. Introduce chronological (time-based) train/test splitting and periodic retraining/drift monitoring to better emulate a deployed predictive-maintenance setting.
8. Containerize the application (e.g., Docker) and add a hosted/cloud deployment path with authentication, since no deployment configuration currently exists (Section 5.6, Section 15.1).
9. Integrate real-time streaming ingestion (e.g., MQTT/Kafka) so the dashboard's Live Prediction page can score a continuous sensor feed instead of manual entry or batch CSV upload only.
10. Add alerting integration (e.g., email/SMS to maintenance teams) triggered by high failure probability, and a persistent database/audit trail of past predictions.

21. Conclusion

This project set out to address a concrete predictive-maintenance brief: build an end-to-end pipeline that cleans messy industrial pump telemetry and reliably classifies equipment failure without crashing on the disconnected or corrupted sensor readings that are routine in real IoT deployments. The resulting system reads raw sensor telemetry modelled on the Kaggle “Pump Sensor Data” schema, engineers simple time features, derives a binary equipment_failure target, and feeds a shared, leakage-free scikit-learn preprocessing pipeline into three fairly compared classifiers - Logistic Regression, Random Forest, and Gradient Boosting.

Evaluated by stratified 5-fold cross-validation and a single held-out test split, Logistic Regression was selected as the best-performing candidate by F1-score, achieving 91.95% test accuracy, 83.20% recall, and 0.9395 ROC-AUC on the minority failure class, at the cost of a comparatively low 25.68% precision - a trade-off consistent with prioritising catching real failures over minimising false alarms. The model's coefficients meaningfully concentrate on the sensors the underlying data-generating process was designed to make predictive, giving reasonable confidence that the pipeline is learning genuine signal rather than artefacts.

Practically, the project demonstrates that a carefully engineered classical ML pipeline - with no novel algorithmic contribution - can still deliver real value in this domain through disciplined engineering: leakage-free preprocessing, fair multi-model comparison, and, most distinctively, a prediction layer whose resilience to malformed input was iteratively debugged and hardened rather than assumed. The system should be regarded as a well-built prototype and demonstrator: it has not been validated on real plant data, uses fixed hyperparameters, and its current precision/recall trade-off would need threshold tuning before any real deployment. Within that honest scope, it achieves what it set out to do - a pipeline that never crashes on bad sensor input and a documented, reproducible comparison of three standard classifiers for pump-failure prediction.

22. References

- [1] F. Pedregosa et al., “Scikit-learn: Machine Learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [2] W. McKinney, “Data Structures for Statistical Computing in Python,” in *Proc. 9th Python in Science Conf. (SciPy 2010)*, Austin, TX, USA, 2010, pp. 56–61.
- [3] C. R. Harris et al., “Array Programming with NumPy,” *Nature*, vol. 585, pp. 357–362, 2020.
- [4] Streamlit Inc., “Streamlit Documentation.” [Online]. Available: <https://docs.streamlit.io> (accessed 2026).
- [5] Plotly Technologies Inc., “Plotly Python Open Source Graphing Library.” [Online]. Available: <https://plotly.com/python/> (accessed 2026).
- [6] Joblib Developers, “Joblib: Running Python Functions as Pipeline Jobs.” [Online]. Available: <https://joblib.readthedocs.io> (accessed 2026).
- [7] N. Phantawee, “Pump Sensor Data,” Kaggle. [Online]. Available: <https://www.kaggle.com/datasets/nphantawee/pump-sensor-data>. Note: this is the real-world dataset whose column schema the project targets (timestamp, sensor_00...sensor_NN, machine_status); the results reported in this case study were produced on the project's own schema-compatible synthetic generator (generate_sample_data.py), not on this source dataset directly - see Section 6.
- [8] J. D. Hunter, “Matplotlib: A 2D Graphics Environment,” *Computing in Science & Engineering*, vol. 9, no. 3, pp. 90–95, 2007. (Used to render this document's figures from the project's stored results.)